czii-cryo-et-object-identification

4th Place Solution [Source Codes & Submission Notebook Released!]

Rank	4
Team	yu4u & tattaka
Author	yu4u
Topic ID	561401
Version	Original

Source: https://www.kaggle.com/competitions/czii-cryo-et-object-identification/discussion/561401
Retrieved with Kaggle CLI on 2026-10-21.

We would first like to express our gratitude to the competition host and the Kaggle staff for organizing this outstanding competition. Below, we introduce the solution of Team yu4u & tattaka.

Summary

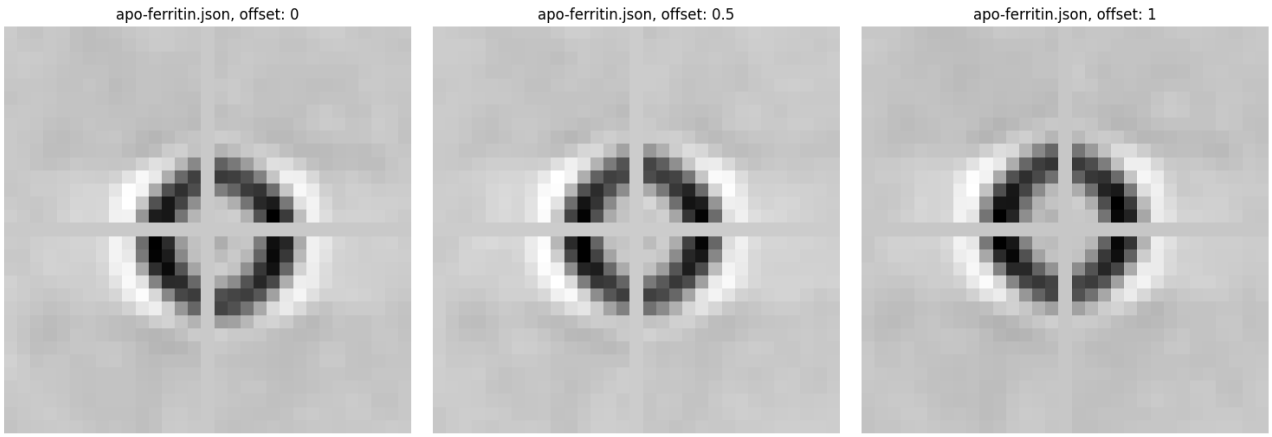
We adopted an approach to detect particle points using a heatmap-based method, which is the most commonly employed technique in pose estimation and facial keypoint detection.
Since this competition deals with 3D images rather than 2D images, we utilized two types of UNet-like models (yu4u's model and tattaka's model) that take 3D voxels as input and outputs 3D heatmaps.

Our Approach to This Competition

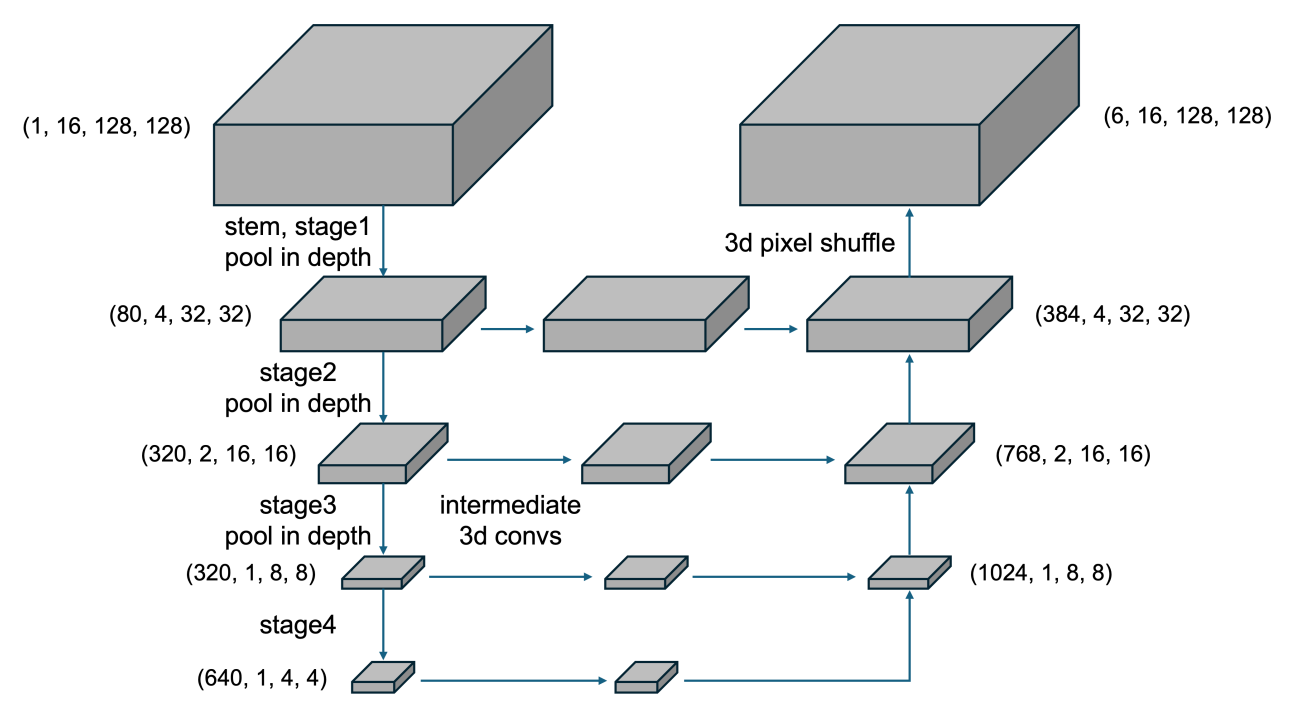
First, we will explain our approach to this competition, specifically how we addressed the issue of CV and LB not correlating. We used CV only to confirm that the metric produced was somewhat reasonable and for selecting checkpoints. For models with potential for improvement, we simply submitted them and relied on the LB to make decisions about which methods to adopt or discard.

Creating the Ground Truth Heatmap

We generate the ground truth heatmap necessary for model training. This involves converting the ground truth particle coordinates into the pixel coordinate system and creating a mask using a Gaussian function, where the particle center is set to 1.0 and sigma is 6 pixels for yu4u's model. For tattaka's model, different sigma values were used for different particles based on their sizes.
We believe that an offset of 1.0 should be added when converting particle coordinates into the pixel coordinate system. While this discussion suggests adding 0.5, our notebook demonstrates that 1.0 is the correct value. The main difference is that the previous discussion assumes the particle center is at the top-left of a pixel, whereas we argue that, on average, the circle should be drawn from the pixel center (0.5, 0.5).



yu4u's Model



We adopted a 2.5D-UNet, which utilizes a 2D image-based model as the backbone. The outputs from each stage of this backbone are pooled along the depth direction, enabling hierarchical feature extraction in the depth dimension as well. This idea was borrowed from the excellent notebook. An interesting observation is that replacing this pooling operation with strided 3D convolutions degrades performance. This would be because the pooling method effectively aggregates depth features while preserving the original 2D backbone's feature maps as much as possible. Similar to many other Kaggle competitions dealing with 3D data, a UNet utilizing a 2D backbone outperformed a straightforward UNet with a 3D backbone.

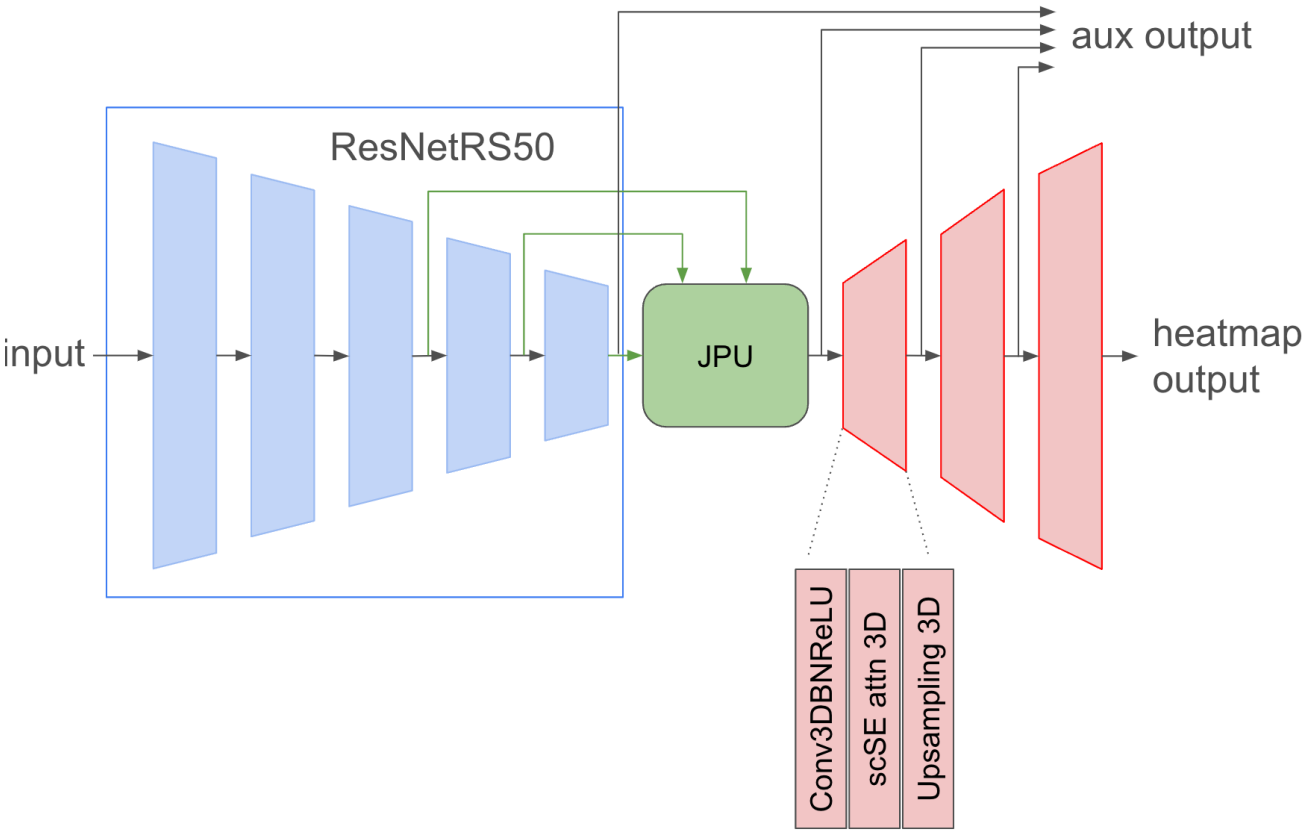
We also applied 3D convolution between the encoder and decoder, inspired by the 3rd Place Solution of the contrails competition.

Initially, we used a plain UNet architecture, but processing high-resolution feature maps required significant memory and computation. To address this, we adopted a model that outputs the final heatmap using pixel shuffle from a feature map with a stride of 4. Pixel shuffle, also known as depth_to_space in TensorFlow, is an operation that redistributes information from the channel dimension to the spatial dimensions. Compared to deconvolution, it offers advantages in computational efficiency and reducing artifacts.

For the final submission, we used four models with different folds of a ConvNeXt Nano model as the backbone.

tattaka's Model

Model Architecture



This model is a lightweight 2.5D UNet with ResNetRS-50 as the backbone.

The input to the model is a volume of size 32×128×128 (D×H×W), and it outputs a 3D heatmap of the same size. Within the backbone, the depth is progressively reduced by half using average pooling for the first two stages. After that, average pooling with kernel=3, stride=1, padding=1 is used to maintain the depth while computing feature maps. As a result, the feature map shapes at each stage of the backbone are as follows: `(bs, ch, 16, 64, 64)`, `(bs, ch, 8, 32, 32)`, `(bs, ch, 8, 16, 16)`, `(bs, ch, 8, 8, 8)`, `(bs, ch, 8, 4, 4)`.

In the decoder, the three lowest-resolution feature maps are fed into Joint Pyramid Upsampling. These maps are then progressively upsampled using 3D CNNs, SESC attention, and upsampling layers until they reach the same size as the input volume.

Loss Function

Since the number of particles within the volume is relatively small, there is a significant class imbalance between positive and negative samples during training. We attempted to adjust the parameters for generating ground truth heatmaps, but this did not lead to any improvement in cross-validation performance.

Ultimately, we implemented a simple MSE-based loss function to balance positive and negative samples, which allowed for faster convergence:

```
loss = MeanSquaredError(pred, true)

pos_loss = (loss * true).() / (true.() + )
neg_loss = (loss * ( - true)).() / (( - true).() + )

balanced_loss = pos_loss + neg_loss
```

Inference Tips

Finally, we used four yu4u's models and three tattaka's models in the final submission.
To stay within the time limit, we optimized our models by converting them to TensorRT format for faster inference. The conversion process was based on this notebook.
Additionally, we selected a Kaggle Notebook instance with dual T4 GPUs and leveraged multiprocessing to parallelize inference.

Post Processing

For the final heatmap, we first detect local maxima using non-maximum suppression, which is implemented via max pooling with a kernel size of 7. Next, the detected points are filtered using different thresholds for each particle type.

Since the detected points are in the pixel coordinate system, we need to convert them into the particle coordinate system. To do so, we proceed as follows:

1. **Centering**: Add 0.5 to the pixel coordinates to shift from the pixel's top-left to its center.
2. **Offset Correction**: Subtract the 1.0 offset that was added during heatmap generation.
3. **Scaling**: Multiply by 10.012 to convert the adjusted pixel coordinates to the particle coordinate system.

Does Not Work for Us

- Two-stage model: We built a model that refines the scores by cropping regions around the points detected using a heatmap approach and then applying a classification model to those cropped regions. Although it worked well in terms of CV scores, it did not improve the LB performance.

Source Code and Notebooks

- Final submission

- Training tattaka's model
- Training yu4u's model

Supplemental Q&A

> **suguuuuu · 2025-02-06T00:17:37.677000**
>
> Congratulations!!!!!!
>
> I have three questions:
>
> 1. Compared to the standard MSE, Focal loss, BCE, did the loss function you used have a different effect to performance?(How much difference?)
> 2. You set the kernel size to 7. Is there any reason you didn't make it variable based on the size of the evaluation particles?
> 3. You are using max pooling. Is there any reason for not using average pooling?

> **tattaka · 2025-02-06T00:44:03.773000**
>
> 1. Initially, I experimented with Focal Loss, BCE, and FBeta Loss for training, but the MSE-based loss ultimately yielded better final accuracy. Additionally, when using EMA for smoothing, it is crucial to balance positive and negative samples; otherwise, convergence slows down significantly.
> 2. In the host notebook, the kernel size was dynamically adjusted for each particle. We considered implementing the same approach, but due to its lower priority and limited submission attempts, I was unable to test it.
> 3. Max pooling in post-processing serves as a peak detection mechanism.